Field Hours

CSCE 665 Advanced Networking and Security

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

States before payloads

You treat a network protocol as a state machine plus a message format. That pair is the mechanism. Each state names which messages are legal, and each message has a fixed set of fields. A toy handshake uses 3 states: idle, sent, and established. A data record that shows up in idle is not application data, and you drop it. You do not open this course before CS 144, because you need those field names first.

On a live machine you keep the bytes and the state in one note. The mechanism is a numbered packet tied to a state name. Packet 4 while the endpoint is idle is a protocol error, even when the payload parses. You store 1 capture per trial and you hash that file before you filter it. A diagram that has states but no packet numbers cannot be checked against the machine.

A record is legal only in some states.

1. Name the current state
2. Check the message type
3. Drop data while idle
4. Hash the capture

Packet 4 arrives too early

1. The endpoint is idle, the first of the 3 states. Packet 4 parses as a data record.
2. You drop it. Established is the only state in this toy where application data is legal.
3. You hash the capture before any filter. The note says packet 4 and idle, or it is not checkable.

Problem 1. A data record arrives and your note says the endpoint is idle. What do you do with it, and how many handshake states does this toy define?

Answer. You drop it. The toy has 3 states: idle, sent, and established. Data is not legal in idle. You also hash the 1 capture before you filter it.

Problem 2. Your diagram lists the three states and no packet numbers. What can a reader not check?

Answer. They cannot check the diagram against the machine. The mechanism is a packet number tied to a state name, such as packet 4 in idle. You were also supposed to finish CS 144 before these questions.

Lecture 2 · 90 minutes

The tag covers every field that moves

You authenticate a message with a tag computed over chosen bytes and a shared key. The mechanism is recompute and compare. Suppose the tag is 16 bytes. You accept the message only when all 16 bytes match the tag you just computed. A match on the first 15 bytes and a miss on the last byte is a reject. You do not hand the payload to the next state after a partial match.

You include every field that must not be swapped. Destination, sequence number, and payload are 3 inputs to the tag in this toy. If you leave the sequence number out, a tag can be valid for the payload and still sit on the wrong packet. A check over 2 of the 3 fields is a different protocol. You write down which fields you covered. The byte length of the tag does not repair a field you skipped.

Partial agreement is still a failed tag.

All 16 bytes match

Recompute the tag

Compare every byte

Accept and move state

15 match, 1 misses

The last byte differs

Reject the message

Do not deliver payload

Three fields, or it is another protocol

1. You compute the tag over destination, sequence, and payload. That is 3 inputs, and the tag is 16 bytes.

2. All 16 bytes match, so this message can move to the next check, which is freshness.

3. A second design that omits the sequence number is not this protocol. You would write that it covers 2 fields, not 3.

Problem 1. The recomputed tag matches on 15 bytes and differs on 1. The tag length is 16. Do you accept?

Answer. No. All 16 bytes must match. A partial match is a reject, and you do not hand the payload to the next state.

Problem 2. The tag covers destination and payload, and it omits the sequence number. How many of the 3 required inputs are missing?

Answer. 1 input, the sequence number. A valid tag on the other 2 fields can still sit on the wrong packet. Tag length does not repair the skipped field.

Lecture 3 · 90 minutes

A counter makes a copy stale

A valid tag does not make a message fresh. Replay is acceptance of the same bytes twice. The mechanism is a counter the receiver stores. When the last accepted sequence is 10, the next message must present 11 or higher. Sequence 10 with a valid tag is a replay, and you drop it. You persist the counter across a reboot. A restart that sets the counter back to 0 will accept old messages again.

You keep a reorder window so a late packet is not punished. A window of width 4, after sequence 10, admits 11, 12, 13, and 14 in any order, each one time. Sequence 10 stays outside that window. You store 4 bits, one per slot. A second copy of sequence 12 finds its bit already set, and you drop the copy. The bit, not the tag alone, is what makes the second copy stale.

Freshness sits on top of a valid tag.

Accept only a full tag

Keep the last sequence

Admit a window of 4

Persist the counter

Sequence 12 arrives twice

1. The last accepted sequence is 10. The window width is 4, so 11 through 14 are eligible once each.

2. The first copy of sequence 12 has a valid tag. You set its bit among the 4 bits and you accept it.

3. The second copy still has a valid tag, but the bit is set. You drop it. Sequence 10 remains outside the window.

Problem 1. The last accepted sequence is 10. A message with sequence 10 arrives with a valid 16-byte tag. What do you do?

Answer. You drop it. It is a replay. The next sequence must be 11 or higher. A valid tag is not freshness, and a reboot must not reset the stored counter to 0.

Problem 2. After sequence 10, the window width is 4 and sequence 12 has already been accepted. How many slots does the window have, and do you accept a second 12?

Answer. The window has 4 slots, for 11, 12, 13, and 14. You do not accept the second 12, because its bit is already set. Sequence 10 stays outside.

Lecture 4 · 90 minutes

A chain is not a banner

You authenticate a server with a certificate chain, not with a name printed in a banner. Each certificate is a signature over the next public key. A chain of length 3 contains a leaf, one intermediate, and a root you already trust. You verify 3 signatures, and you check that the leaf name is the name you asked for. If the chain ends at an intermediate you do not trust, you fail the handshake even when the inner signatures verify.

The session key is not that long-term signing key. After the 3 checks succeed, you derive a session key and you tag records with it. A valid record tag does not, by itself, tell you which server you reached. You needed the chain for that. In this toy the handshake times out at 5 seconds, and a peer that never finishes remains in the sent state. You send no application data while the state is still sent.

Three signatures, then a session key.

Check the leaf name -> Verify 3 signatures -> Require a trusted root -> Derive the session key

The third link never reaches a root

1. The chain you received has a leaf and one intermediate. It does not reach a root you stored. Length so far is 2, not 3.

2. Both signatures you can see do verify. You still fail the handshake, because the chain does not end at a trust anchor.

3. Five seconds pass in the sent state with no completion. You send no application data, and you do not treat a later record tag as server authentication.

Problem 1. You can verify 2 signatures, and the chain stops at an intermediate you do not trust. How many links is a full chain in this toy, and do you continue?

Answer. A full chain has 3 links: leaf, intermediate, and a root you trust. You do not continue. Inner signatures do not replace the missing trust anchor or the leaf-name check.

Problem 2. The handshake has been in the sent state for 5 seconds and then a record arrives with a valid session tag. What is missing?

Answer. A finished chain check. The toy timeout is 5 seconds, and you send no application data from the sent state. A record tag does not name the server. The chain of 3 did that job.

Lecture 5 · 90 minutes

An answer must match the question

A resolver accepts an answer only when it matches the question you sent. The mechanism is a comparison of 3 fields: the name, the type, and the transaction id. You asked for name N, type A, and id 42. An answer that carries id 42 but a different name is not your answer, and you drop it. You also refuse to cache a record you did not ask for, even if it shared that packet.

You also honor the time to live, which is a count of seconds in the record. If that count is 30, you may reuse the address until the timer reaches 0, and then you ask again. A client that keeps the address for 300 seconds is ignoring the record. On the live machine you log 3 columns: the id, the name, and the stored TTL. Those columns explain a later connection without inventing packets.

Cache a record only after it matches.

1. Compare name and type

2. Compare transaction id

3. Drop extra records

4. Honor the TTL

Id 42, wrong name

1. Your question is name N, type A, id 42. The response repeats id 42 and type A, but the name is different.

2. You drop that answer. You also skip a second record in the same packet that you never asked for.

3. A matching record with TTL 30 may be reused for 30 seconds. You log id, name, and 30. You do not keep it for 300.

Problem 1. You asked for name N with id 42. The packet has id 42 and a different name, plus an address you never requested. What do you cache?

Answer. Nothing from that packet. The name fails the 3-field match, and you do not cache a record you did not ask for even when it shares the packet.

Problem 2. The matching record says TTL 30. Your client still holds the address at 300 seconds. By how many seconds did it overrun the record?

Answer. 270 seconds. 300 minus 30 is 270. You may reuse the address only until the timer hits 0, then you ask again. Log the id, the name, and the TTL you actually stored.

Lecture 6 · 90 minutes

First match is the policy

A packet filter is an ordered rule list, and the mechanism is first match. You start at rule 1 and you stop at the first predicate that matches. If rule 1 accepts any source headed for destination port 22, then rule 4, which would deny one source, never sees that port. Order is the policy. A deny you append as rule 6 does not affect packets an earlier accept already took.

You write each predicate with 4 fields you can point at: source, destination, port, and action. A missing port means the rule is wider than the prose you wrote beside it. On a live machine you log the rule number that fired, not only the word accept. If 80 of 100 packets hit rule 1, the later rules are almost idle, so inspect rule 1 first. New rules do not narrow a rule above them.

Walk rules until one predicate hits.

Start at rule 1 -> Test the four fields -> Stop at first match -> Log that rule number

Rule 1 hides rule 4

1. Rule 1 accepts any source to destination port 22. Rule 4 would deny one particular source on that port.

2. A packet to port 22 matches rule 1. You stop. Rule 4 does not run, and you log rule 1.

3. In a sample of 100 packets, 80 hit rule 1. You inspect rule 1 before you add rule 7.

Problem 1. Rule 1 accepts destination port 22 from any source. Rule 4 denies one source on port 22. Which rule fires for that source, and which rule never runs?

Answer. Rule 1 fires and rule 4 never runs. First match stops the walk. Appending the deny as rule 6 would not change packets rule 1 already accepted.

Problem 2. A sample has 100 packets and 80 of them hit rule 1. How many packets hit something else, and what do you inspect first?

Answer. 20 packets hit something else, because 100 minus 80 is 20. You inspect rule 1 before adding rule 7. Log the rule number, and write all 4 fields: source, destination, port, and action.

Lecture 7 · 90 minutes

A snapshot is not a timeline

You separate the socket table from the packet capture when you investigate a live host. The table is a snapshot with 4 columns: local address, remote address, state, and process. The capture is only the interval you recorded. A socket that is established, with no packet inside a 10-minute capture, can still be an idle connection. You print the start time and the stop time next to the snapshot so the gap is visible.

You correlate the two sources only where address and time can meet. If the capture starts at minute 15 and you sampled the socket table at minute 12, the hole is 3 minutes. You do not invent packets for those minutes. Byte totals are sums you can recompute: flows of 40 bytes and 60 bytes total 100. You still name each flow. One unlabeled total hides which remote address you actually observed.

Two sources with different clocks of validity.

Socket snapshot

Four columns, one instant

Sampled at minute 12

Idle sockets may be quiet

Packet capture

Only while it was running

Starts at minute 15

Hash it before a filter

Forty bytes plus sixty

1. The socket table was sampled at minute 12. The capture starts at minute 15, so the hole is 3 minutes.

2. Inside the capture, two flows account for 40 bytes and 60 bytes. You report 100 and you name both flows.

3. An established socket with no packet in the 10-minute capture stays an idle connection, not a contradiction you paper over.

Problem 1. The socket sample is at minute 12 and the capture starts at minute 15. How long is the hole, and may you fill it with packets?

Answer. The hole is 3 minutes, because 15 minus 12 is 3. You do not invent packets for it. Write both times next to the 4-column snapshot.

Problem 2. Two flows show 40 bytes and 60 bytes. What total do you report, and what do you still have to name?

Answer. 100 bytes, because 40 plus 60 is 100. You still name each flow. One unlabeled total hides the remote address.

Lecture 8 · 90 minutes

What you can now check on a trace

You can now check a protocol on a live machine as states, tags, and windows. Data that arrives in the idle state, 1 of 3 states, is dropped. A 16-byte tag must match on every byte, and sequence 10 is a replay once 10 has been accepted. A chain of 3 certificates is not a banner. First match means rule 1 can hide rule 4. You keep the capture hash and the minute window beside any claim.

The public lab for this material is MIT 6.858, and you take it after CS 144, not before. These 8 lectures are the checks you should already be able to name: state, tag coverage, freshness, chain length, name match, and rule order. Your closing exercise is 1 capture of 10 minutes plus 1 socket snapshot. If you cannot say which rule fired, or you cannot recompute 40 plus 60, this course is not finished.

Checks you should be able to name in order.

State, then message type

Full tag, then freshness

Chain of 3, then session

First matching filter rule

One capture, one sum

1. You hash a 10-minute capture and you place a socket snapshot beside it, with the 3-minute hole written out if the times differ.

2. Two flows of 40 and 60 bytes sum to 100. You name both, and you name the rule number that accepted them.

3. You can point at idle versus established, at a 16-byte tag, and at sequence 10 as a replay. The lab course is 6.858, after CS 144.

Problem 1. Flows of 40 bytes and 60 bytes were accepted by rule 1, and rule 4 would have denied one of them. What is the byte total, and which rule do you cite?

Answer. The total is 100, because 40 plus 60 is 100. You cite rule 1, because first match never reached rule 4. If you cannot say which rule fired, this course is not finished.

Problem 2. Sequence 10 was already accepted, the tag is 16 bytes and matches, and the endpoint is established. Is the message fresh?

Answer. No. Sequence 10 is a replay once 10 has been accepted. The matching tag only gets the message to the freshness check. Data would also have been dropped if the state were still idle, 1 of 3 states.